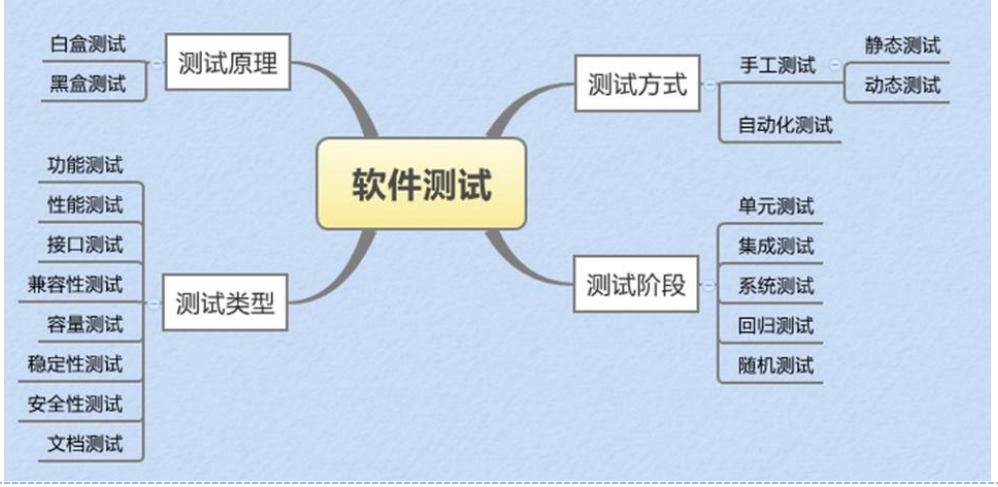


软件工程
软件测试

陈龙彪
厦门大学
2025年秋学期

软件测试的目的

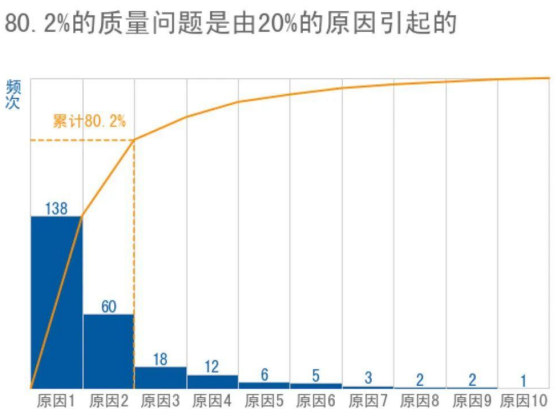
- Glen Myers给出的软件测试目的：
  - 测试：为了发现错误而执行程序的过程
  - 好的测试用例：可能找到迄今为至尚未发现的错误
  - 成功的测试：揭示迄今为至尚未发现的错误
- 测试可用于证明软件的功能和性能与需求说明相符合
- 测试结果数据可以为可靠性分析提供依据





测试原则

1. 所有的测试都应可追溯到客户需求
2. 应该在测试工作真正开始前的较长时间就进行测试计划
3. Pareto原则：测试发现的80%的错误可能来自于20%的程序代码
4. 测试应从小规模开始，逐步转向大规模
5. 穷举测试是不可能的
6. 为了达到最有效的测试，应由独立第三方来承担测试



帕累托法则（英语：Pareto principle，也被称为 80/20 法则、关键少数法则、八二法则）指出，**约仅有20%的变因操纵着80%的局面**。也就是说：所有变量中，最重要的仅有20%，虽然剩余的80%占了多数，控制的范围却远低于“关键的少数”。80/20的概念是意大利经济学家 帕累托 (Vilfredo Pareto) 在洛桑大学发现的，并于他的第一篇文章《政治经济学》中说明了该现象，例如：意大利约有80％的土地由20％的人口所有、80%的豌豆产量来自20%的植株等等。

其他测试原则

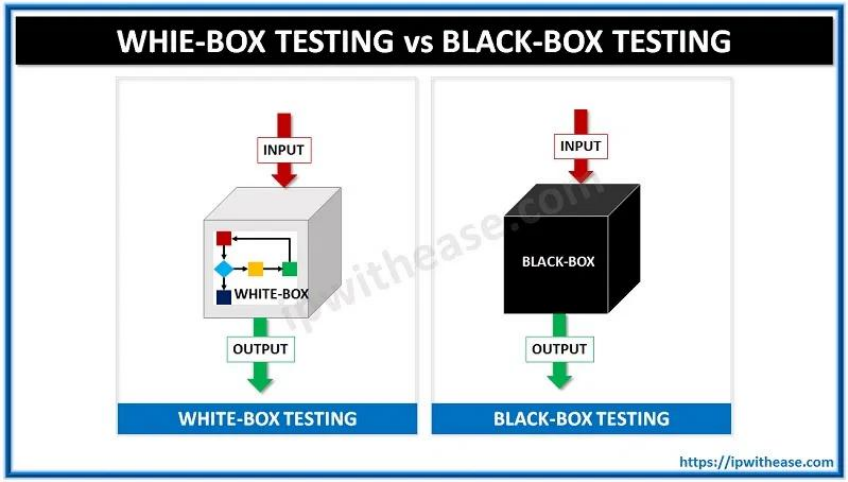
1. 设计测试用例时，应同时包括合理和不合理的输入条件
2. 严格执行测试计划，排除测试的随意性
3. 应当对每一个测试结果做全面检查
4. 妥善保存测试计划、测试用例、出错统计和最终分析报告，为维护提供方便
5. 检查程序是否做了应做的事仅是成功的一半，另一半是检查程序是否做了不该做的事
6. 在规划测试时不要设想程序中不会查出错误

测试用例设计

- 测试用例的设计是软件测试的关键所在
- 设计尽可能少的测试用例来发现尽可能多的错误
- 设计最有可能发现软件错误的测试用例
- 避免使用发现错误效果相同的测试用例
- 测试用例的设计方法可分为两类：白盒测试和黑盒测试

测试用例设计

- 白盒测试：知道产品的内部工作过程，通过测试来检验产品内部动作是否按照规格说明书的规定正常进行
- 黑盒测试：知道产品应该具有的功能，通过测试来检验每个功能是否都能正常使用



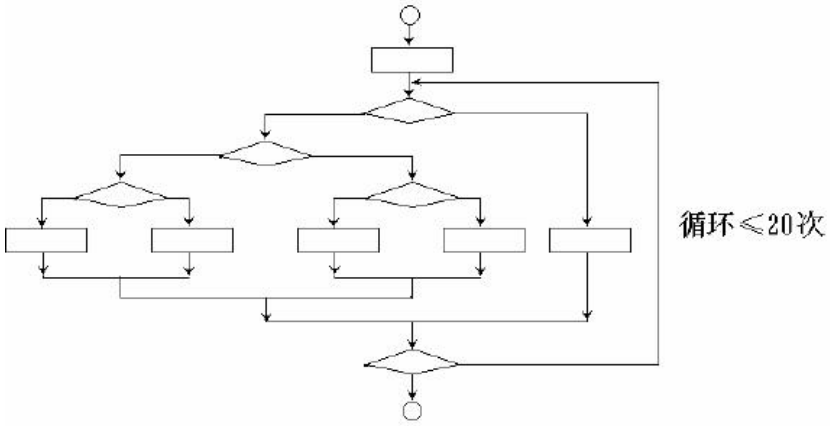
黑盒测试	白盒测试
不涉及程序结构	考查程序逻辑结构
用软件规格说明生成测试用例	用程序结构信息生成测试用例
确认测试和系统测试	单元测试和集成测试
某些代码段得不到测试	对所有逻辑路径进行测试

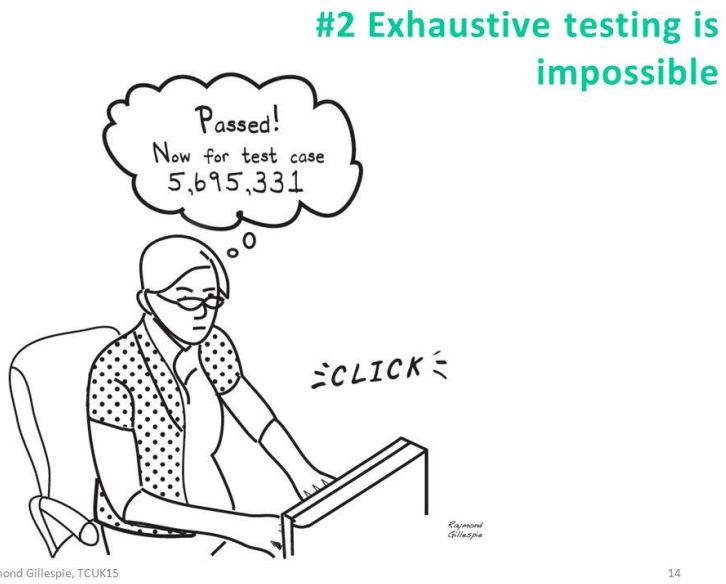
白盒测试

- 白盒测试把测试对象看作一个透明的盒子，测试人员根据程序内部的逻辑结构及有关信息设计测试用例，检查程序中所有逻辑路径是否都按预定的要求正确地工作
- 白盒测试主要用于对模块的测试，包括：
  - 程序模块中的所有独立路径至少执行一次
  - 对所有逻辑判定的取值都至少测试一次
  - 在上下边界及可操作范围内运行所有循环
  - 测试内部数据结构的有效性等

白盒测试 vs. 穷举测试

- 对一个具有多重选择和循环嵌套的程序，不同的路径数目可能是天文数字
  - 例如一个小程序的流程图，它包括了一个执行20次的循环，其循环体有五个分支。这个循环的不同执行路径数达520条，如果对每一条路径进行测试需要1毫秒，那么即使一年工作365×24小时，要想把所有路径测试完，大约需3170年





白盒测试

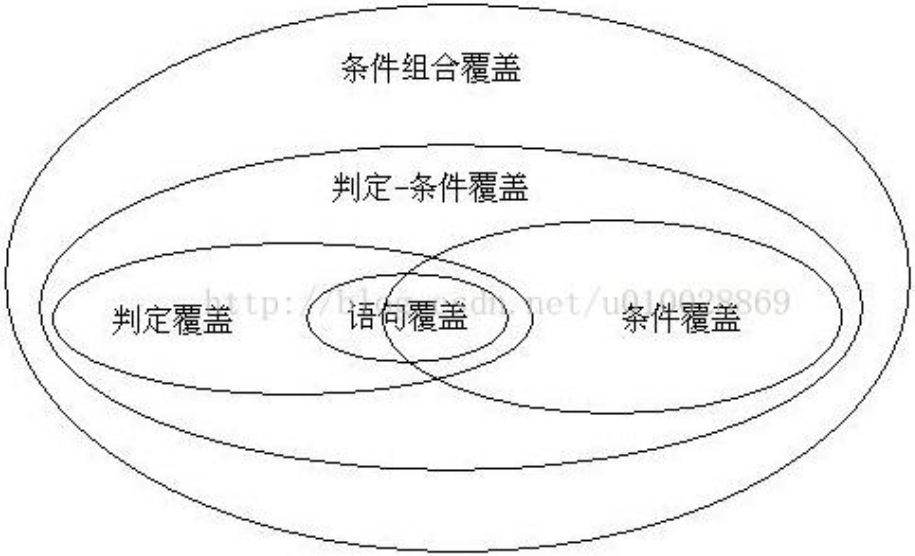
- 常用的白盒测试方法有：
  - 逻辑覆盖测试
  - 基本路径测试
  - 数据流测试
  - 循环测试

白盒测试

- 常用的白盒测试方法有：
  - 逻辑覆盖测试
  - 基本路径测试
  - 数据流测试
  - 循环测试

逻辑覆盖测试

- 逻辑覆盖主要考察使用测试数据运行被测程序时对程序逻辑的覆盖程度。通常希望选择最少的测试用例来满足所需的覆盖标准。主要的覆盖标准有：
  1. 语句覆盖
  2. 判定覆盖
  3. 条件覆盖
  4. 判定-条件覆盖
  5. 条件组合覆盖
  6. 路径覆盖

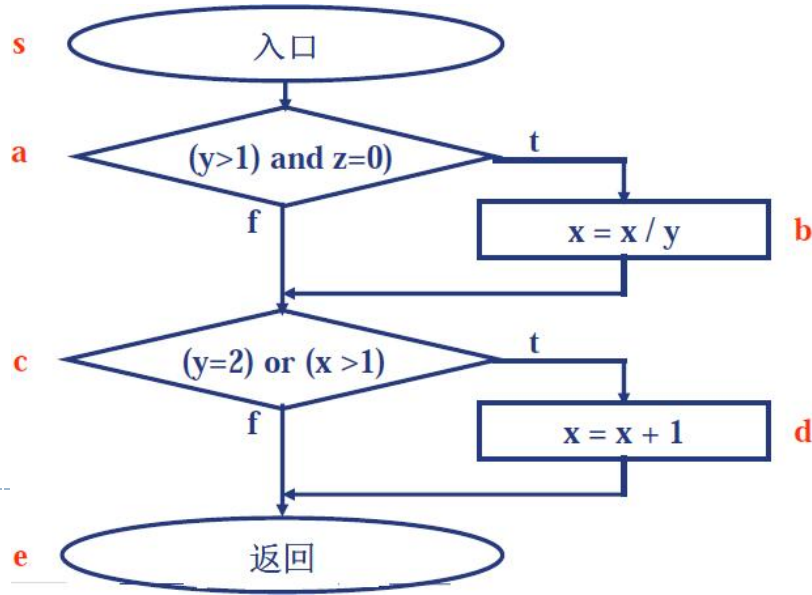


逻辑覆盖测试

- 例：对下列子程序进行测试
  - 该子程序接受x、y、z的值，并将计算结果x的值返回给调用程序

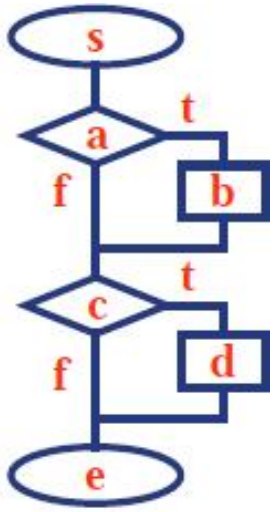
```
procedure example(y:real, z:real, x:real);
begin
    if (y>1) and (z=0)        then x:=x/y;
    if (y=2) or (x>1)         then x:=x+1;
end;
```

- 与该子程序对应的流程图如下



1 语句覆盖

- 语句覆盖是指选择足够的测试用例，使得运行这些测试用例时，被测程序的每个可执行语句都至少执行一次
  - 例：欲使每个语句都执行一次，只需执行路径sabcde即可
- 测试用例（y，z，x）：
  - 输入（2，0，4），输出（2，0，3）
  - 是否能发现：and->or？

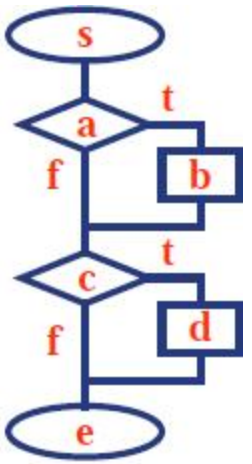


a: (y>1) and (z=0)
c: (y=2) or (x>1)

2 判定覆盖

- 判定覆盖（也称分支覆盖）是指选择足够的测试用例，使得运行这些测试用例时，被测程序的每个判定分支都至少执行一次
- 测试用例（y, z, x）：
  - 路径：sacde+sabce；sabcde+sace

测试数据	预期结果	路径	a	c
2，1，1	2,1,2	sacde	f	t
3，0，3	3,0,1	sabce	t	f



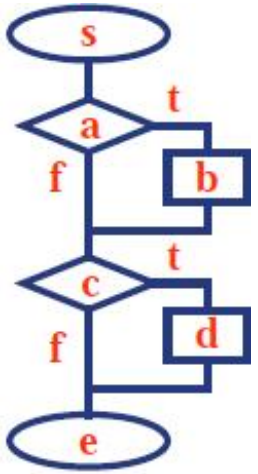
a: (y>1) and (z=0)
c: (y=2) or (x>1)

- 判定覆盖使得程序中的所有语句也必定都至少执行一次；因此满足判定覆盖标准的测试用例也一定满足语句覆盖标准

3 条件覆盖

- 条件覆盖是指选择足够的测试用例，使得运行这些测试用例时，被测程序的每个判定中条件的所有可能值都至少出现一次
- 测试用例（y, z, x）：

测试数据	预期结果	路径	覆盖的条件
1,0,3	1,0,4	sacde	FTFT
2,1,1	2,1,2	sacde	TFTF



a: (y>1) and (z=0)
c: (y=2) or (x>1)

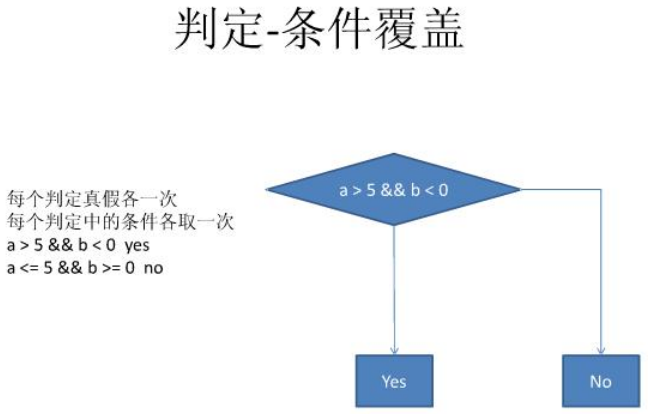
3 条件覆盖

- 与语句覆盖的关系
  - 虽然每个条件的所有可能值都出现过，但判定表达式的某些可能结果并未出现
  - 仅满足条件覆盖有时无法满足语句覆盖
- 与判定覆盖的区别
  - 关心条件取值，不关心判定结果
  - 判定覆盖不一定包含条件覆盖，条件覆盖也不一定包含判定覆盖

4 判定–条件覆盖

- 判定-条件覆盖是指选择足够的测试用例，使得运行这些测试用例时，每个判定的所有可能结果都至少执行一次，并且每个判定中条件的所有可能值都至少出现一次
- 显然，满足判定-条件覆盖标准的测试用例一定也满足判定覆盖、条件覆盖、语句覆盖标准

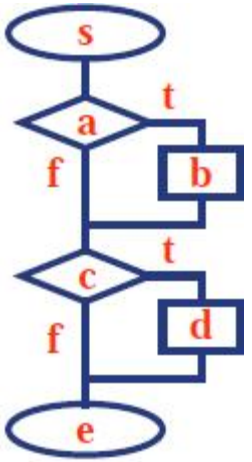
例：**y>1 and z=0**
一组符合判定/条件覆盖的用例需要满足：
1. 使得判定为真
2. 使得判定为假
3. 使得y>1
4. 使得y<=1
5. 使得z=0
6. 使得z!=0



5 条件组合覆盖

- 条件组合覆盖是指选择足够的测试用例，使得运行这些测试用例时，被测程序的每个判定中条件的各种可能组合都至少出现一次
- 满足条件组合覆盖标准的测试用例一定也满足判定覆盖、条件覆盖、判定-条件覆盖、语句覆盖标准
- 例：测试用例（y, z, x）：

测试数据	预期结果	路径	覆盖的条件
2,0,4	2,0,3	sabcde	TTTT
2,1,1	2,1,2	sacde	TFTF
1,0,3	1,0,4	sacde	FTFT
1,1,1	1,1,1	sace	FFFF

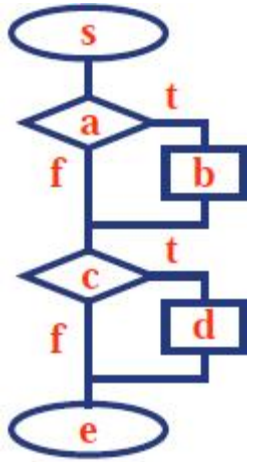


a: (y>1) and (z=0)
c: (y=2) or (x>1)

5 条件组合覆盖

- 条件组合覆盖是上述五种覆盖标准中最强的，但条件组合覆盖仍不能保证程序中所有可能的路径都被覆盖。
  - 本例中，满足条件组合覆盖标准的测试用例就没有经过sabce路径

测试数据	预期结果	路径	覆盖的条件
2,0,4	2,0,3	sabcde	TTTT
2,1,1	2,1,2	sacde	TFTF
1,0,3	1,0,4	sacde	FTFT
1,1,1	1,1,1	sace	FFFF

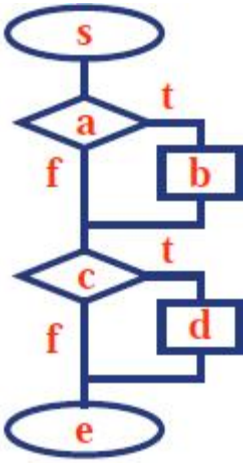


a: (y>1) and (z=0)
c: (y=2) or (x>1)

6 路径覆盖

- 路径覆盖是指选择足够的测试用例，使得运行这些测试用例时，被测程序的每条可能执行到的路径都至少经过一次
  - 如果程序中包含环路，则要求每条环路至少经过一次
- 测试用例(y, z, x)：

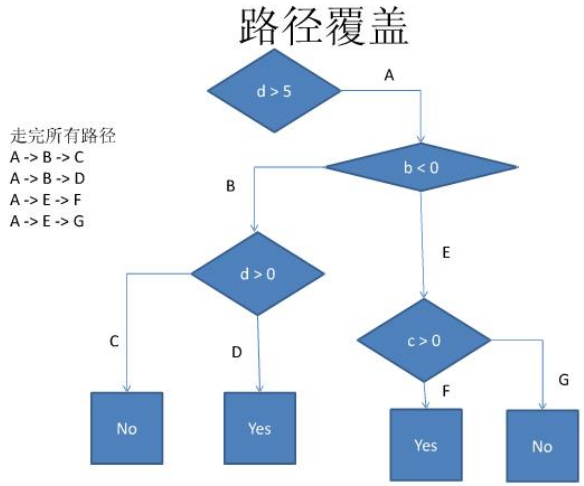
测试数据	预期结果	路径	覆盖的条件
2,0,4	2,0,3	sabcde	TTTT
1,1,1	1,1,1	sace	FFFF
1,0,2	1,0,3	sacde	FTFT
3,0,3	3,0,1	sabce	TTFF

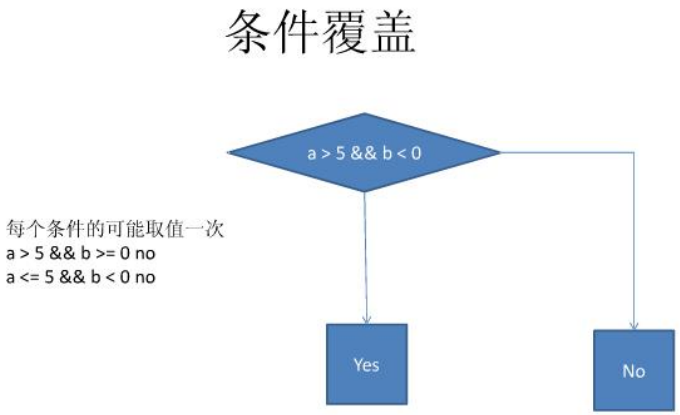


a: (y>1) and (z=0)
c: (y=2) or (x>1)

6 路径覆盖

- 路径覆盖实考虑了程序中各种判定结果的所有可能组合，但它未必能覆盖判定中条件值的各种可能情况。因此，它是一种比较强的覆盖标准，但不能替代条件覆盖和条件组合覆盖





一个例子

```
if A and B then Action1
if C or D then Action2
```

①语句覆盖最弱，只需要让程序中的语句都执行一遍即可 。

上例中只需设计测试用例使得A=true B=true C=true 即可。

②分支覆盖又称判定覆盖：使得程序中每个判断的取真分支和取假分支至少经历一次，即判断的真假均曾被满足。

上例需要设计测试用例使其分别满足下列条件即可（1）A=true，B=true，C＝true，D=false（2）A=true，B=false，C＝false，D=false。

③条件覆盖：要使得每个判断中的每个条件的可能取值至少满足一次。

上例中第一个判断应考虑到A=true，A=false，B=true，B=false第二个判断应考虑到C＝true，C＝false，D=true，D=false，所以上例中可以设计测试用例满足下列条件（1）A=true，B=true，C＝true，D=true（2）A=false，B=false，C＝false，D=false。

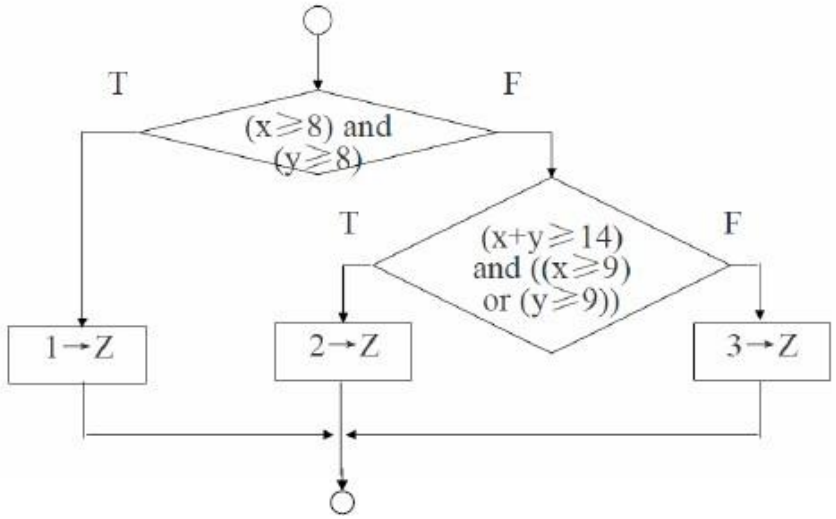
④路径覆盖：要求覆盖程序中所有可能的路径。

所以可以设计测试用例满足下列条件（1）A=true，B=true，C＝true，D=true（2）A=false，B=false，C＝false，D=false（3）A=true，B=true，C＝false，D=false（4）A=false，B=false，C＝true，D=true。

练习1

- 对下面的流程图采用逻辑覆盖测试技术，可供选择的测试用例已给出，请选择其中尽量少的测试用例构成用例组，以分别满足判定覆盖、条件覆盖、判定-条件覆盖、条件组合覆盖、路径覆盖

	x	y
I	3	9
II	4	9
III	5	5
IV	7	7
V	7	9
VI	9	3
VII	9	7
VIII	9	9
IX	8	7
X	8	8



白盒测试

- 常用的白盒测试方法有：
  - 逻辑覆盖测试
  - 基本路径测试
  - 数据流测试
  - 循环测试

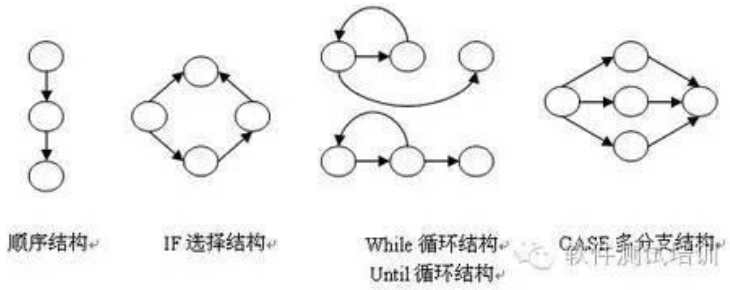


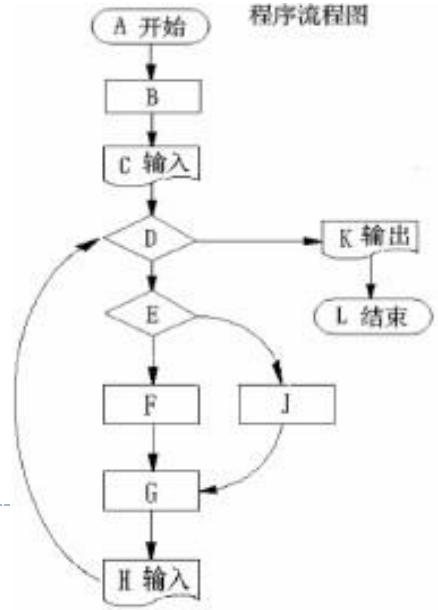
基本路径测试

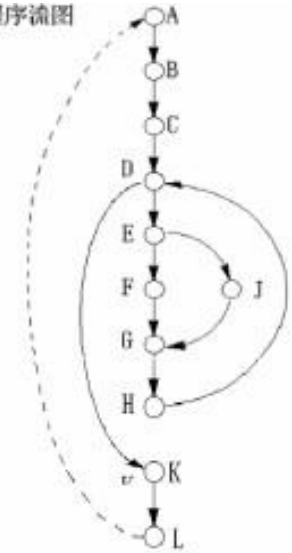
- 在实际问题中，一个不太复杂的程序，特别是包含循环的程序，其路径数可能非常大。因此测试常常难以覆盖程序中的所有路径，为此，我们希望把测试的程序路径数压缩到一定的范围内
- 基本路径测试是Tom McCabe提出的一种白盒测试技术。这种方法在程序控制流图的基础上，通过分析控制构造的环复杂度，导出基本可执行路径集合，最后为每一条基本路径设计一个测试用例

控制流图

- 程序控制流图由结点和边组成，分别用圆和箭头表示
- 设计图中一个连续的处理框（对应于程序中的顺序语句）序列和一个判定框（对应于程序中的条件控制语句）映射成流图中的一个结点
- 设计图中的箭头（对应于程序中的控制转向）映射成流图中的一条边
- 对于设计图中多个箭头的交汇点可以映射成流图中的一个结点（空结点）

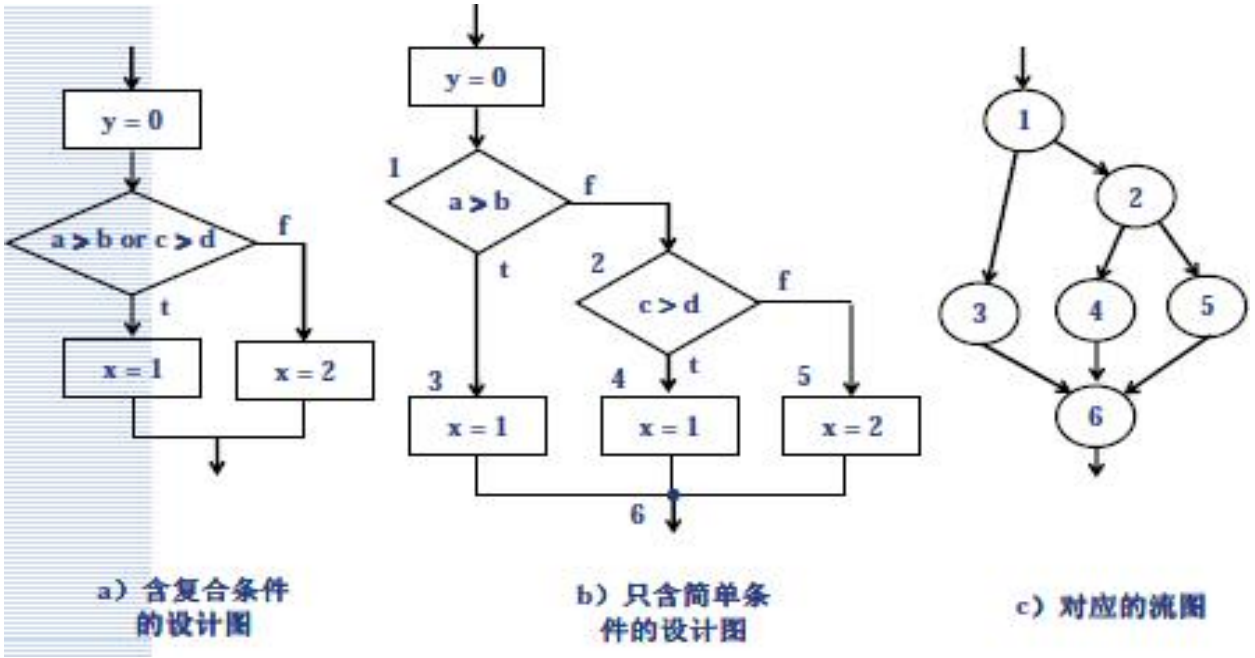






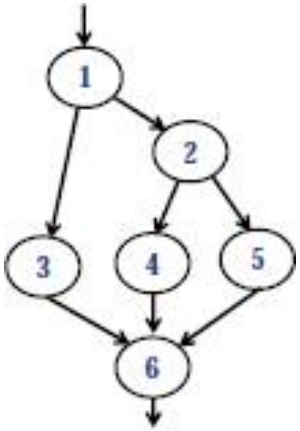
控制流图

- 上述映射的前提是设计图的判定中不包含复合条件。如果设计图的判定中包含了复合条件，那么必须先将其转换成等价的简单条件设计图



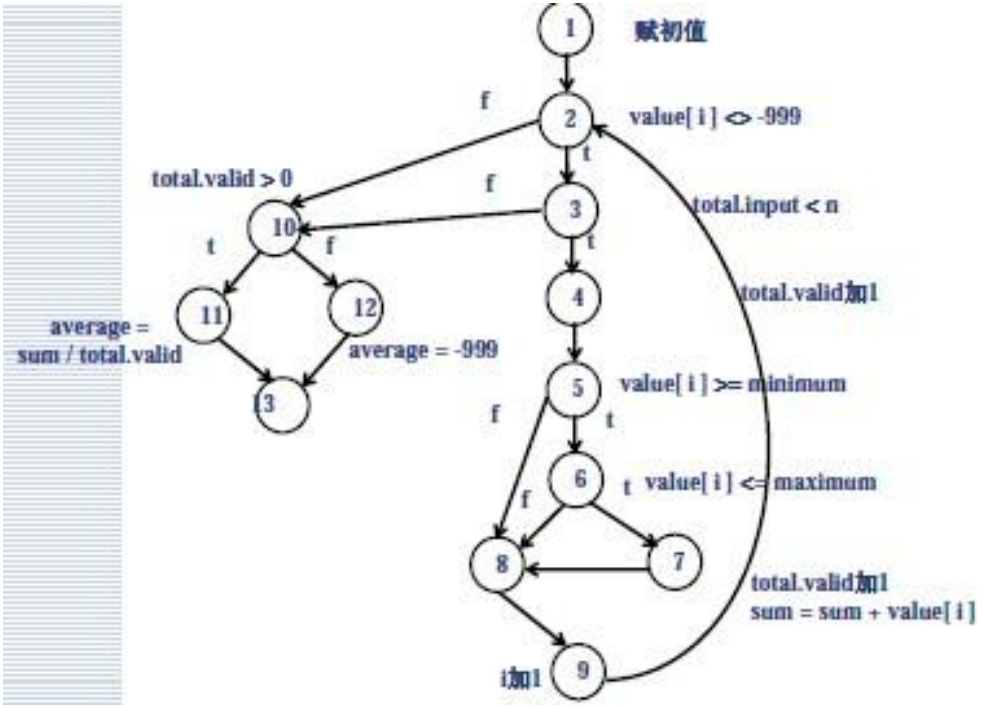
独立路径

- 流图中由结点和边组成的闭合部分称为一个区域 (region)
  - 在计算区域数时，图的外部部分也作为一个区域。例如下图所示的流图的区域数为3
- 独立路径是指程序中至少引进一个新的处理语句序列或一个新条件的任一路径。在基本路径测试时，独立路径的数目就是流图的区域数



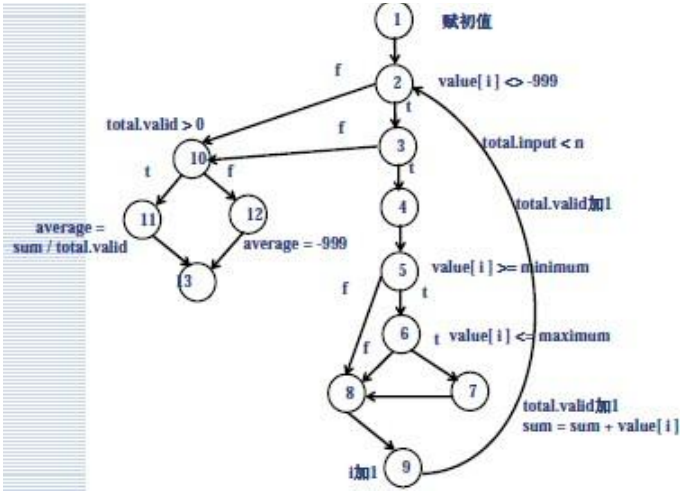
基本路径测试示例

- 例如，对一个程序进行基本路径测试，该程序的功能是：最多输入N个值（以-999为输入结束标志），计算位于给定范围内的那些值（称为有效输入值）的平均值，以及输入值的个数和有效值 的个数



基本路径测试示例

- 其区域数为6，选取独立路径如下：
  - 路径1：1-2-10-11-13
  - 路径2：1-2-10-12-13
  - 路径3：1-2-3-10-11-13
  - 路径4：1-2-3-4-5-8-9-2-10-12-13
  - 路径5：1-2-3-4-5-6-8-9-2-10-12-13
  - 路径6：1-2-3-4-5-6-7-8-9-2-10-11-13
- 为每一条独立路径设计测试用例
  - 假设：n = 5；minimum = 0；maximum = 100



基本路径测试示例

- 路径1：1-2-10-11-13
  - 测试数据：value = [90，-999，0，0，0]
  - 预期结果：Average = 90，total.input = 1, total.valid = 1
- 路径2：1-2-10-12-13
  - 测试数据：value = [-999 ，0，0，0，0]
  - 预期结果：Average = -999，total.input = 0，total.valid = 0
- 路径3：1-2-3-10-11-13
  - 测试数据：value = [-1，90，70，-1，80]
  - 预期结果：Average = 80，total.input = 5，total.valid = 3

基本路径测试示例

- 路径4：1-2-3-4-5-8-9-2-10-12-13
  - 测试数据：value = [-1，-2，-3，-4，-999]
  - 预期结果：Average = -999，total.input = 4，total.valid = 0
- 路径5：1-2-3-4-5-6-8-9-2-10-12-13
  - 测试数据：value = [120，110，101，-999，0]
  - 预期结果：Average = -999，total.input = 3，total.valid = 0
- 路径6：1-2-3-4-5-6-7-8-9-2-10-11-13
  - 测试数据：value = [95，90，70，65，-999]
  - 预期结果：Average = 80，total.input = 4，total.valid = 4

练习2

- 使用基本路径测试方法，为下面程序设计测试用例

```
01:  start
     input (a,b,c,d)
02: if (a>0)
03: and (b>0)
04: then x=a+b
05: else x=a-b
06: endif
07: if (c>a)
08: or (d<b)
09: then y=c-d
10: else y=c+d
11: endif
12: print (x,y)
13: stop
```

步骤提示

1. 根据PDL画出程序控制流图
2. 计算区域数
3. 确定独立路径
4. 为每条独立路径设计测试用例

白盒测试

- 常用的白盒测试方法有：
  - 逻辑覆盖测试
  - 基本路径测试
  - 数据流测试
  - 循环测试

数据流测试

- 数据流测试是根据程序中变量的定义和引用位置来选择测试用例的方法
- 假定s为语句的标号（每个语句有唯一的标号），x为变量名。定义：
  - DEF（s）= { x | 语句s中含有对x的定义 }
  - USE（s）= { x | 语句s中含有对x的引用 }
  - 当s为分支或循环语句时，DEF（s）= f
- 设变量x在语句s中被定义，如果存在一条从语句s到语句s′的路径，并且在这条路径上不存在对x的其它定义，则称变量x在s处定义在s′处仍有效

数据流测试

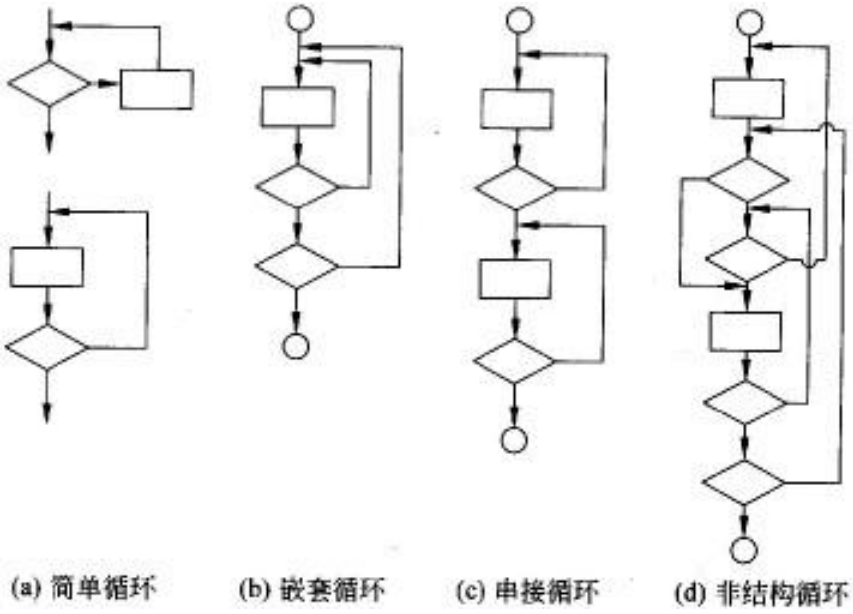
- 定义：定义-引用链DU
  - 变量x的定义-引用链为[x, s, s′]
  - 其中s, s′为语句标号,
    - x ∈ DEF (s) ∩ USE (s′)
    - 且s处定义的x 在s′处仍有效
- 数据流测试设计测试用例使得**每个DU链至少被覆盖一次**
- 数据流测试适用于嵌套IF和多重循环程序的测试

白盒测试

- 常用的白盒测试方法有：
  - 逻辑覆盖测试
  - 基本路径测试
  - 数据流测试
  - 循环测试

循环测试

- 循环分为4种不同类型：简单循环、嵌套循环、串接循环和非结构循环
- (1) 简单循环
  - ① 零次循环：从循环入口到出口
  - ② 一次循环：检查循环初始值
  - ③ 二次循环：检查多次循环
  - ④ m次循环：检查多次循环
  - ⑤ 最大次数循环
  - ⑥ 比最大次数多一次的循环
  - ⑦ 比最大次数少一次的循环



循环测试

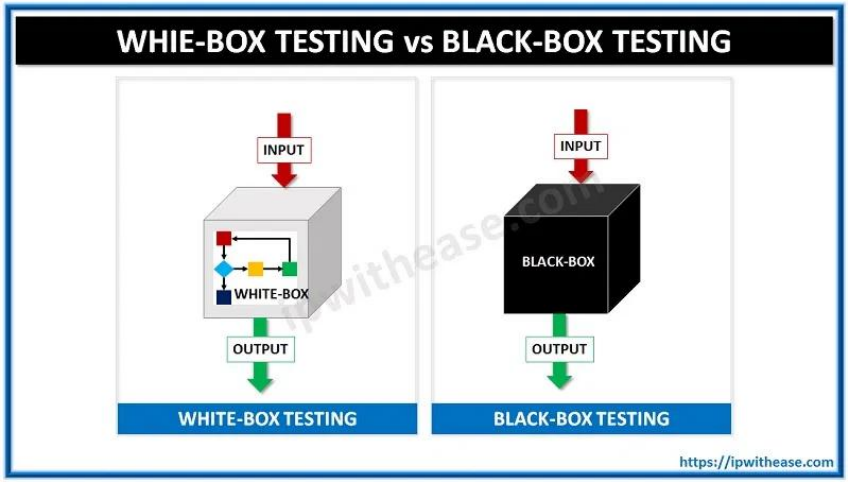
- （2）嵌套循环
  - ① 先测试最内层循环：所有外层的循环变量置为最小值，由最内层按简单循环测试
  - ② 由里向外，测试上一层循环：测试时此层以外的所有外层循环的循环变量取最小值，此层以内的所有嵌套内 层循环的循环变量取“典型”值，该层按简单循环测试
  - ③ 重复上一条规则，直到所有各层循环测试完毕
  - ④ 对全部各层循环同时取最小循环次数，或同时取最大循环次数

循环测试

- (3) 串接循环
  - 如果串接的各个循环互相独立，则可以分别用简单循环的方法进行测试；但如果第一个循环的循环变量与第二个循环控制相关，则两个循环不独立，此时，把第一个循环看作外循环，第二个循环看作内循环，然后用测试嵌套循环的办法来处理
- (4) 非结构循环
  - 这一类循环应该先将其结构化，然后再测试

测试用例设计

- 白盒测试：知道产品的内部工作过程，通过测试来检验产品内部动作是否按照规格说明书的规定正常进行
- 黑盒测试：知道产品应该具有的功能，通过测试来检验每个功能是否都能正常使用



黑盒测试	白盒测试
不涉及程序结构	考查程序逻辑结构
用软件规格说明生成测试用例	用程序结构信息生成测试用例
确认测试和系统测试	单元测试和集成测试
某些代码段得不到测试	对所有逻辑路径进行测试

黑盒测试

- 黑盒测试是依据软件的需求规约，检查程序的功能是否符合需求规约的要求
- 主要的黑盒测试方法有：
  1. 等价类划分
  2. 边界值分析
  3. 比较测试
  4. 错误猜测
  5. 因果图
  6. 功能图



1 等价类划分

- 由于不能穷举所有可能的输入数据进行测试，所以只能选择少量有代表性的输入数据，来揭露尽可能多的程序错误
- 等价类划分方法将所有可能的输入数据划分成若干等价类，然后在每个等价类中选取一个代表性数据作为测试用例
  - 等价类是指输入域的某个子集，该子集中的每个输入数据对揭露软件中的错误都是等效的，测试等价类的某个代表值就等价于对这一类其他值的测试
  - 也就是说，如果该子集中的某个输入数据能检测出某个错误，那么该子集中的其他输入数据也能检测出同样的错误；反之，如果该子集中的某个输入数据不能检测出错误，那么该子集中的其他输入数据也不 能检测出错误

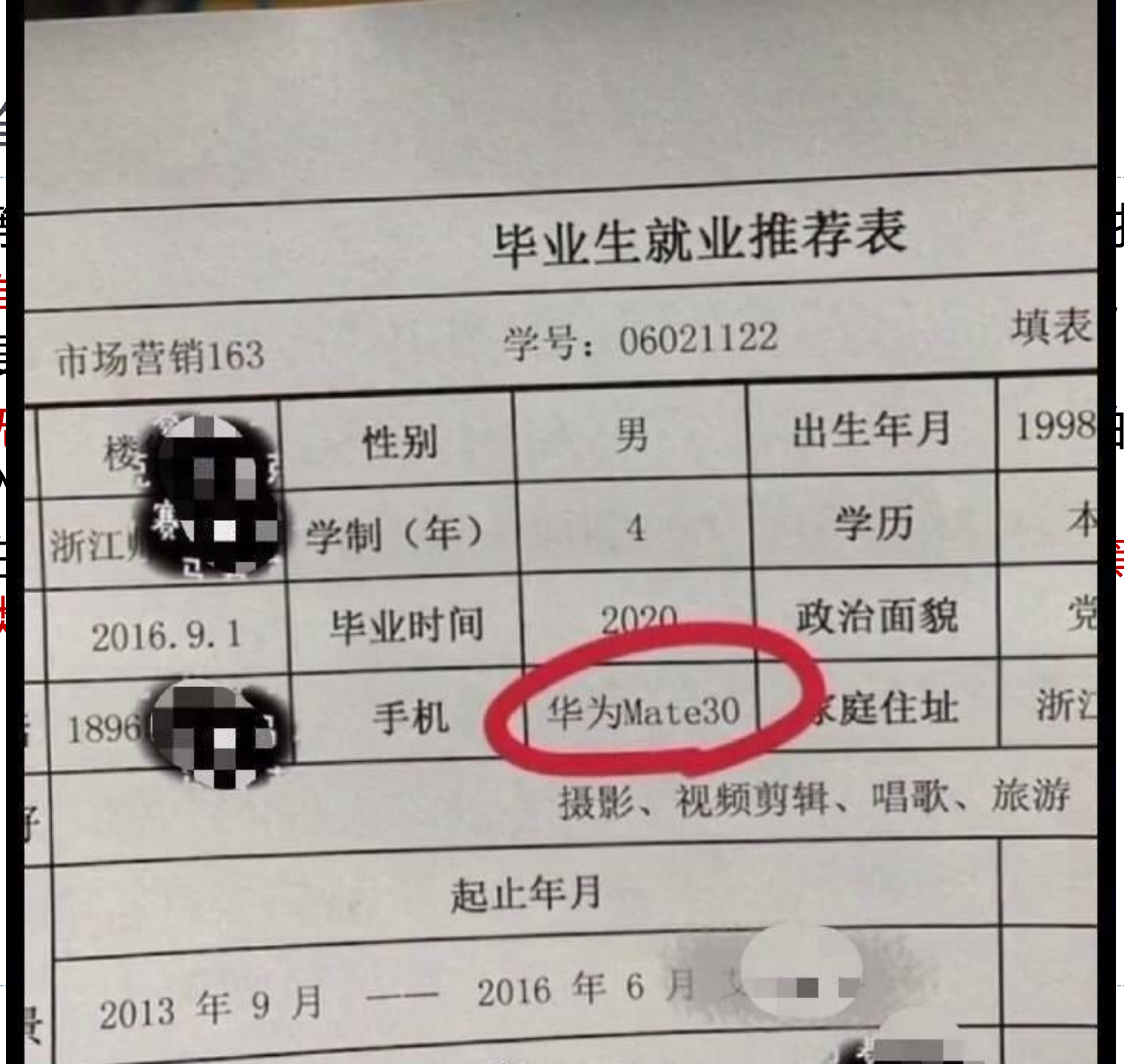
输入等价类	有效等价类	无效等价类
日期的类型及长度	①6 位数字字符	②有非数字字符 ③少于 6 位数字字符 ④多于 6 位数字字符
年份范围	⑤在 1990~2049 之间	⑥小于 1990 ⑦大于 2049
月份范围	⑧在 01~12 之间	⑨等于 00 ⑩大于 12

1 等价类划分

- 等价类划分把输入数据分为有效输入数据和无效输入数据
- 有效输入数据指符合规格说明要求的合理的输入数据，主要用来检验程序是否实现了规格说明中的功能
- 无效输入数据指不符合规格说明要求的不合理或非法的输入数据，主要用来检验程序是否做了规格说明以外的事
- 在确定输入数据等价类时，常常还要分析输出数据的等价类，以便根据输出数据等价类导出输入数据等价类

1全

- 等 据
- 有 主
- 要
- 无 的输
- 入
- 在 等价
- 类



毕业生就业推荐表

市场营销163		学号：06021122		填表
楼[redacted]	性别	男	出生年月	1998
浙江[redacted]	学制（年）	4	学历	本
2016.9.1	毕业时间	2020	政治面貌	党
1896[redacted]	手机	华为Mate30	家庭住址	浙江
	摄影、视频剪辑、唱歌、旅游			
	起止年月			
	2013 年 9 月 —— 2016 年 6 月			

等价类划分设计测试用例的步骤

- 确定等价类
  - 根据软件的规格说明，对每一个输入条件（通常是规格说明中的一句话或一个短语）确定若干个有效等价类和若干个无效等价类。
  - 可使用如下表格

输入条件	有效等价类	无效等价类

等价类划分设计测试用例的步骤

- 确定等价类的规则：
  1. 如输入条件规定了取值范围，则可确定一个有效等价类（输入值在此范围内）和两个无效等价类（输入值小于最小值及大于最大值）

输入等价类	有效等价类	无效等价类
日期的类型及长度	①6 位数字字符	②有非数字字符 ③少于 6 位数字字符 ④多于 6 位数字字符
年份范围	⑤在 1990~2049 之间	⑥小于 1990 ⑦大于 2049
月份范围	⑧在 01~12 之间	⑨等于 00 ⑩大于 12

  2. 如输入条件规定了值的个数，则可以确定一个有效等价类（输入值的个数等于规定的个数）和两个无效等价类（输入值的个数小于规定的个数和大于规定的个数）
     - 例如，规定输入构成三角形的3条边，则有效等价类是“输入边数= 3”，无效等价类是“输入边数< 3”和“输入边数> 3”

等价类划分设计测试用例的步骤

- 确定等价类的规则：
  3. 如输入条件规定了输入值的集合（即离散值），且程序对不同的输入值做不同的处理，那么每个允许的值都确定为一个有效等价类，另外还有一个无效等价类（任意一个不允许的值）
     - 例如，规定输入的考试成绩为优、良、中、及格、不及格，则可确定5个有效等价类和一个无效等价类
  4. 如果输入条件规定了输入值必须遵循的规则，那么可确定一个有效等价类（符合此规则）和若干个无效等价类（从各个不同的角度违反此规则）
     - 例如，对变量标识符规定为“以字母开头的……串”。那么有效等价类是“以字母开头的串”，而无效等价类有“以数字开头的串”、“以标点符号开头的串”等

等价类划分设计测试用例的步骤

- 确定等价类的规则：
  5. 如果输入条件规定输入数据是整型，那么可以确定三个有效等价类（正整数、零、负整数）和一个无效等价类（非整数）
  6. 如果输入条件规定处理的对象是表格，那么可以确定一个有效等价类（表有一项或多项）和一个无效等价类（空表）

等价类划分设计测试用例的步骤

- 设计测试用例
  - 在确定了等价类之后，建立等价类表，列出所有划分出的等价类，并为每个有效等价类和无效等价类编号
  - 设计一个新的测试用例，使其尽可能多地覆盖尚未被覆盖的有效等价类，重复这一步，直到所有的有效等价类都被覆盖为止
  - 为每个无效等价类设计一个新的测试用例

输入条件	有效等价类	无效等价类

输入等价类	有效等价类	无效等价类
日期的类型及长度	①6 位数字字符	②有非数字字符 ③少于 6 位数字字符 ④多于 6 位数字字符
年份范围	⑤在 1990~2049 之间	⑥小于 1990 ⑦大于 2049
月份范围	⑧在 01~12 之间	⑨等于 00 ⑩大于 12

等价类划分设计测试用例的示例

- 某编译程序的规格说明中关于标识符的规定如下：
  - 标识符是由字母开头，后跟字母或数字的任意组合构成
  - 标识符的字符数为1~8个
  - 标识符必须先说明后使用
  - 一个说明语句中至少有一个标识符
  - 保留字不能用作变量标识符

用等价类划分方法，建立输入等价类表：

输入条件	有效等价类	无效等价类
第一个字符	字母(1)	数字(2) 非字母数字字符(3)
后跟的字符	字母(4) 数字(5)	非字母数字字符(6) 保留字(7)
字符数	1~8个(8)	0个(9) >8个(10)
标识符的使用	先说明后使用(11)	未说明已使用(12)
标识符个数	≥1个(13)	0个(14)

等价类划分设计测试用例的示例

- 选取以下9个测试用例，它们覆盖了所有的等价类

输入数据	预期结果	覆盖的等价类
VAR P3t2：REAL； BEGIN P3t2：=3．1； …… END；	正确标识符	(1)，(4)，(5)，(8)，(11)，(13)
VAR 3P：REAL；	报错：不正确标识符	(2)
VAR ！X：REAL；	报错：不正确标识符	(3)
VAR T#：CHAR；	报错：不正确标识符	(6)

输入数据	预期结果	覆盖的等价类
VAR GOTO：INTEGER；	报错：保留字作标识符	(7)
VAR X，：REAL；	报错：标识符长度为0	(9)
VAR T12345678：REAL；	报错：标识符字符超长	(10)
VAR PAR：REAL； BEGIN …… PAP：=3．14 …… END；	报错：未说明已使用	(12)
VAR ：REAL；	报错：标识符个数为0	(14)

用等价类划分方法，建立输入等价类表：

输入条件	有效等价类	无效等价类
第一个字符	字母(1)	数字(2) 非字母数字字符(3)
后跟的字符	字母(4) 数字(5)	非字母数字字符(6) 保留字(7)
字符数	1~8个(8)	0个(9) ＞8个(10)
标识符的使用	先说明后使用(11)	未说明已使用(12)
标识符个数	≥1个(13)	0个(14)

2 边界值分析

- 边界值分析也是一种黑盒测试方法，是对等价类划分的补充
  - 人们从长期的测试工作经验得知，大量的错误是发生在输入或输出范围的边界上，而不是在输入范围的内部。因此针对各种边界情况设计测试用例，其揭露程序中错误的可能性就更大
  - 这里所说的边界是指在输入等价类和输出等价类的边界上、或稍高于其边界值、或稍低于其边界值的一些特定情况
- 等价类与边界值分析的区别
  - 使用等价类分析方法设计测试用例时，原则上等价类中的任一输入数据都可作为该等价类的代表用作测试用例；而边值分析则是专门挑选那些位于边界附近的值（即正好等于、或刚刚大于、或刚刚小于边界的值）作为测试用例

边界值分析

- 边界值分析
  - 人们从长期的经验中发现，大量的错误往往发生在输入或输出范围的边界上，因此针对各种边界情况设计测试用例，可以查出更多的错误。
  - 这里所说的边界情况是指输入或输出等于或略高于其边界值的情况。
- 等价类与边界值分析
  - 使用等价类划分的方法，只要从每个等价类中的任一输入数据都可以作为测试的代表，而边界值分析则是专门挑选那些位于或刚刚大于、或刚刚小于边界的值作为测试数据。



边界值分析选择测试用例的规则

- 1．如果输入条件规定了值的范围，则选择刚刚达到这个范围的边界的值以及刚刚超出这个范围的边界的值作为测试输入数据。
  - 例如，规定输入的考试成绩在0～100之间，则取0，100，－1，101作为测试输入数据。
- 2．如果输入条件规定了值的个数，则分别选择最大个数、最小个数、比最大个数多1、比最小个数少1的数据作为测试输入数据。
  - 例如，规定一个运动员的参赛项目至少1项，最多3项，那么，可选择参赛项目分别是1项、3项、0项、4项的测 试输入数据

边界值分析选择测试用例的规则

- 3．对每个输出条件使用第1条
  - 例如，输出的金额值大于等于0且小于104，则选择使得输出金额分别为0、9999、－1、10000的输入数据作为测 试数据
- 4．对每个输出条件使用第2条
  - 例如，规定输出的一张发票上，至少有1行内容，至多有5行内容，则选择使得输出发票分别有1行、5行、0行、6 行内容的输入数据作为测试数据
- 5．如果程序的输入或输出是个有序集合，例如，顺序文件、表格，则应把注意力集中在有序集的第1个元素和最后一个元素上

边界值分析选择测试用例的规则

- 6. 如果程序中定义的内部数据结构有预定义的边界，例如，数组的上界和下界、栈的大小，则应选择使得正好达到该数据结构边界以及刚好超出该数据结构边界的输入数据作为测试数据
  - 例如，程序中数组A的下界是10，上界是20，则可选择使得A的下标为10、20、9、21的输入数据作为测试数据
- 7. 发挥你的智慧，找出其他可能的边界条件

边界值分析选择测试用例的规则

- 6. 如果
  - 例如，
- 7. 发挥



定义的边界，例如，则应选择使得
小，则应选择使得A的
则可选择使得A的
为测试数据
条件

边界值分析

- 由于边值分析方法所设计的测试用例更有可能发现程序中的错误，因此经常把边值分析方法与其它设计测试用例方法结合起来使用

练习3

- 请使用等价划分法（黑盒测试）对下面的程序构造测试用例，构造时考虑进行边界值分析。
- 输入三角形三条边的长度值，判断这三条边是否能构成三角形，若能，是否构成直角三角形、等腰三角形或等边三角形。输入边长要求在[1,100]范围内的整数

一个程序员碰到了一个问题，他决定用多线程来解决。现在两个他问题了有...



比较测试（back to back)

- 在现实中，有些软件有很高的可靠性要求，特别是那些可能危及人的生命安全的软件系统，如航空航天控制软件、核电厂控制软件等，其软件可靠性绝对重要。此时，需要冗余的硬件和软件来减少错误发生的可能性
- 通常，可由二支软件开发队伍，根据相同的需求规格说明分别开发二个软件版本，然后，用相同的测试用例对二个版本的软件分别进行测试，比较二个版本软件的测试结果，如果测试结果相同，则可认为二个版本的软件都是正确的，如果测试结果不同，则要分析各个版本，以发现错误的所在。这种测试称为比较测试或称为背靠背测试（back-to-back testing）。大多数情况下，可用自动化工具来进行比较测试

比较测试（back to back)

- 值得注意的是，比较测试并不能保证软件没有错误，如果规格说明本身有错，那么所有的版本都可能反映这种错误
- 另外，如果各个版本产生相同的但都不正确的结果，那么比较测试也无法发现这种错误

错误推测法

- 错误猜测是一种凭直觉和经验推测某些可能存在的错误，从而针对这些可能存在的错误设计测试用例的方法
- 这种方法没有机械的执行步骤，主要依靠直觉和经验
- 错误猜测法的基本思想是：列举出程序中所有可能的错误和容易发生错误的特殊情况，然后根据这些猜测设计测试用例

错误推测法示例

- 例如，测试一个排序子程序，可考虑如下情况：
  - 输入表为空；
  - 输入表只有一个元素；
  - 输入表的所有元素都相同；
  - 输入表已排序。
- 又如，测试二分法检索子程序，可考虑如下情况：
  - 表中只有一个元素；
  - 表长为2n；
  - 表长为2n-1；
  - 表长为2n+1

黑盒白盒方法对比

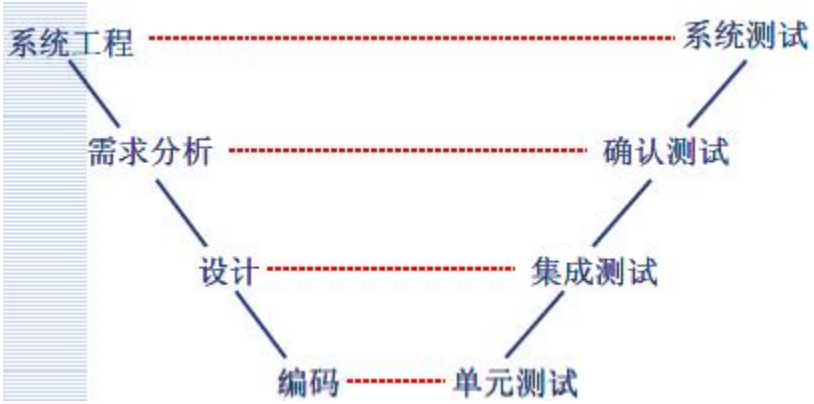
	白盒测试	黑盒测试
优点	①可构成测试数据使特定程序部分得到测试 ②有一定的充分性度量手段 ③可或较多工具支持	①适用于各阶段测试 ②从产品功能角度测试 ③容易入手生成测试数据
缺点	①不易生成测试数据(通常) ②无法对未实现规格说明的部分进行测试 ③工作量大，通常只用于单元测试，有应用局限	①某些代码得不到测试 ②如果规格说明有误，则无法发现 ③不易进行充分性测试
性质	是一种验证技术，回答“我们在正确地构造一个系统吗？”	是一种确认技术，回答“我们在构造一个正确的系统吗？”

测试策略

- 一种测试策略就是将测试分为单元测试、集成测试、确认测试（包括回归测试）和系统测试
  - 单元测试是针对程序中的模块或构件，主要揭露编码阶段产生的错误
  - 集成测试针对集成的软件系统，主要揭露设计阶段产生的错误
  - 确认测试是根据软件需求规约对集成的软件进行确认，主要揭露不符合需求规约的错误
  - 回归测试就是对已经进行过的测试的子集的重新执行，以确保对程序的改变和修改，没有传播非故意的副作用。
  - 对于基于计算机系统中的软件，还需将它集成到基于计算机系统中，并进行系统测试，以揭露不符合系统工程中对软件要求的错误

测试策略

- V模型：描述软件开发各阶段与测试策略之间的对应关系



单元测试（Unit Testing）

- 单元测试又称模块测试，它着重对软件设计的最小单元（软件构件或模块）进行验证
- 单元测试根据设计描述，对重要的控制路径进行测试，以发现构件或模块内部的错误
- 单元测试通常采用白盒测试，并且多个构件或模块可以并行进行测试
- 这里将构件或模块统一称为模块

单元测试的内容

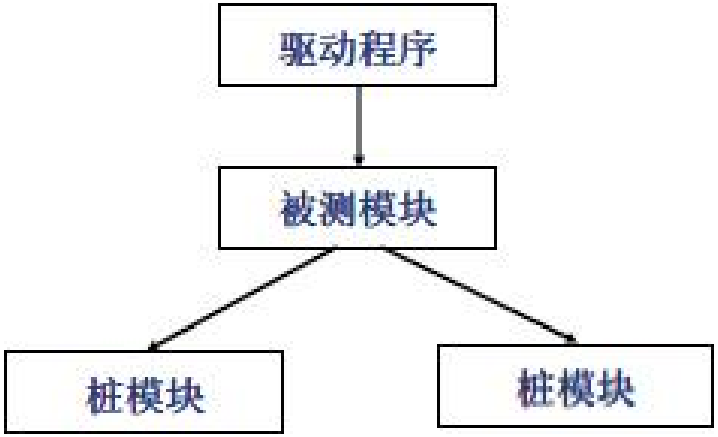
- 模块接口：确保模块的输入/输出参数信息是正确 的。这些信息包括参数的个数、次序、类型等。
- 局部数据结构：确保临时存储的数据在算法执行的整个过程中都能维持其完整性。如不合适的类型说明、不同数据类型的比较或赋值、文件打开和关闭的遗漏、超越数据结构的边界等。
- 边界条件：确保程序单元在极限或严格的情况下仍能正确地执行

单元测试的内容

- 所有独立路径：确保模块中的所有语句都至少执行一次。程序执行的路径实际上体现了计算的过程，计算中常见的错误有：不正确的操作优先级、不同类型数据间的操作、不正确的初始化、不精确的精度、不正确的循环中止、不适当地修改循环变量、发散的迭代等。
- 所有错误处理路径：单元测试应该对所有的错误处理路径进行测试。错误处理部分潜在的错误有：报错信息没有提供足够的信息来帮助确定错误的性质及其发生的位路、报错信息与真正的错误不一致、错误条件在错误处理之前就已引起系统异常、异常条件处理不正确等

单元测试过程

- 单元测试通常与编码工作结合起来进行。
- 模块本身不是一个独立的程序，在测试模块时，必须为每个被测模块开发一个驱动(driver)程序和 若干个桩(stub)模块



单元测试过程

- 驱动模块接收测试数据，调用被测模块，把测试数据传送给被测模块，被测模块执行后，驱动模块接收被测模块的返回数据，并打印相关结果
- 驱动程序的程序结构如下：
  - 数据说明；
  - 初始化；
  - 输入测试数据；
  - 调用被测模块；
  - 输出测试结果；
  - 停止

单元测试过程

- 桩模块的功能是替代被被测模块调用的模块，它接受被测模块的调用，验证入口信息，把控制连同模拟结果返回给被测模块
- 桩模块的程序结构如下：
  - 数据说明；
  - 初始化；
  - 输出提示信息（表示进入了哪个桩模块）；
  - 验证调用参数；
  - 打印验证结果；
  - 将模拟结果送回被测程序；
  - 返回

集成测试（Integrated Testing）

- 集成测试也称组装测试、联合测试
- 经单元测试后，每个模块都能独立工作，但把它们放在一起往往不能正常工作



小明卡里只有8毛钱，本来想打个6毛的饭结果不小心按了9毛的，哪知机器忽然疯狂地喷出255两饭，被喷得满脸热饭的小明大叫“烫烫烫烫烫烫。。。。”

为什么进行集成测试

- 主要问题在于：
  - 数据可能在通过接口时丢失；
  - 一个模块可能对另一个模块产生产生非故意的、有害的影响（即副作用）；
  - 当子功能被组合起来时，可能不能达到期望的主功能；
  - 单个模块可以接受的不精确性（如误差），连接起来后可能会扩大到无法接受的程度；
  - 全局数据结构可能会存在问题

集成测试（Integrated Testing）

- 集成的方式有两种：
  - 非增量式集成：使用“一步到位”的方法来构造程序。 先将所有经过单元测试的模块组合在一起，然后对整个 程序（作为一个整体）进行测试。这种测试在发现错误 时，很难为错误定位。改正错误时容易引入新的错误， 新旧错误混在一起，更难定位
  - 增量式集成：根据程序结构图，按某种次序挑选一个（或一组）尚未测试过的模块，把它集成到已测试好的模块中一起进行测试，每次增加一个（或一组）模块，直至所有模块全部集成到程序中。在增量集成测试过程中发现的错误，往往与新加入的模块有关

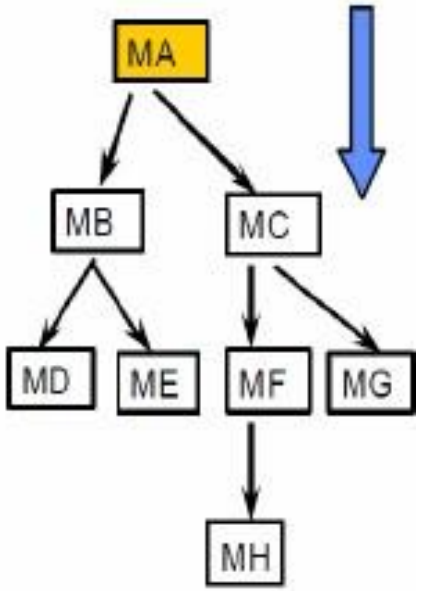
集成测试（Integrated Testing）

- 增量式集成又可分为自顶向下集成和自底向上集成
  - 自顶向下集成：从主控模块（主程序）开始，然后按照程序结构图的控制层次，将直接或间接从属于主控模块的模块按深度优先或广度优先的方式逐个集成到整个结构中，并对其进行测试
  - 自顶向下集成在测试一个模块时，它的上层模块（已测试过）可用作它的驱动模块

集成测试（Integrated Testing）

A possible sequence of integration is:

- Unit test MA
- Integration test MA, MB
- Integration test MA, MB, MC
- Integration test MA, MB, MC, MD
- Integration test MA, MB, MC, MD, ME
- Integration test MA, MB, MC, MD, ME, MF
- Integration test MA, MB, MC, MD, ME, MF, MG
- Integration test all modules



集成测试（Integrated Testing）

- 自顶向下集成的步骤：
  - （1）主控模块（主程序）被直接用作驱动程序，所有直接从属于主控模块的模块用桩模块替换，然后对主控模块进行测试；
  - （2）根据集成的实现方式（深度优先或广度优先），下层的桩模块一次一个地替换成真正的模块，从属于该模块的模块用桩模块替换，然后对其进行测试；
  - （3）用回归测试来保证没有引入新的错误；
  - （4）重复第（2）和第（3）步，直至所有模块都被集成

集成测试（Integrated Testing）

- 自顶向下集成的优点：
  - 不需要驱动模块；能尽早对程序的主要控制和决策机制进行检验，能较早发现整体性的错误；深度优先的自顶向下集成能较早对某些完整的程序功能进行验证。
- 自顶向下集成的缺点：
  - 测试时低层模块用桩模块替代，不能反映真实情况；重要数据不能及时回送到上层模块

集成测试（Integrated Testing）

- 自底向上集成：从程序结构的最底层模块（即原子模块）开始，然后按照程序结构图的控制层次将上层模块集成到整个结构中，并对其进行测试。
- 自底向上集成在测试一个模块时，它的下层模块（已测试过）可用作它的桩模块

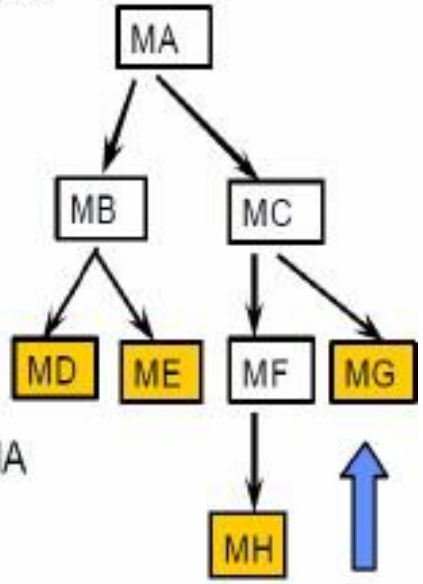
集成测试（Integrated Testing）

- 自底向上集成的步骤：
  - （1）将低层模块组合成能实现软件特定功能的簇；
  - （2）为每个簇编写驱动程序，并对簇进行测试；
  - （3）移走驱动程序，用簇的直接上层模块替换驱动程序，然后沿着程序结构的层次向上组合新的簇；
  - （4）凡对新的簇测试后，都要进行回归测试，以保证没有引入新的错误；
  - （5）重复第（2）步至第（4）步，直至所有的模块都被 集成

集成测试（Integrated Testing）

A possible sequence of integration is:

- Unit test MD
- Unit test ME
- Unit test MH
- Unit test MG
- Integration test MD, MB
- Integration test MD, MB, ME
- Integration test MD, MB, ME, MA
- Integration test MH, MF
- Integration test MH, MF, MC
- Integration test MH, MF, MC, MG
- Integration test all modules



集成测试（Integrated Testing）

- 自底向上集成的优点：
  - 不需要桩模块，所以容易组织测试；将整个程序结构分解成若干个簇，对同一层次的簇可并行进行测试，可提高效率
- 自底向上集成的缺点：
  - 整体性的错误发现得较晚

集成测试（Integrated Testing）

- 策略的选择
  - 自顶向下集成测试与自底向上集成测试各有优缺点，其中一种策略的优点差不多就是另一种策略的缺点。将这两种策略组合起来可能是一种最好的折衷，这种折衷的策略是：在程序结构的高层使用自顶下向策略，而在低层则使用自底向上策略，这种测试策略也称为三明治测试（sandwich testing）
  - 集成测试时应特别关注关键模块（critical module）的测 试。关键模块是指具有下列一个或多个特征的模块：1） 与多个软件需求有关；2）含有高层控制（位于程序结构 的高层）；3）本身是复杂的或是容易出错的；4）含有 确定的性能需求。关键模块应尽早测试，回归测试时也 应集中在关键模块的功能上

回归测试（Regression Testing）

- 在集成测试过程中，每当增加一个（或一组）新模块时，原先已集成的软件就发生了改变。新的数据流路径被建立，新的I/O操作可能出现，还可 能激活新的控制逻辑，这些改变可能使原本正常 的功能产生错误
- 当测试时发现错误后，需修改程序；或者在软件维护时也需修改程序。这些对程序的修改也可能使原本正常的功能产生错误
- 回归测试就是对已经进行过的测试的子集的重新执行，以确保对程序的改变和修改，没有传播非故意的副作用

回归测试（Regression Testing）

- 回归测试集（已经过测试的子集）包括三种不同类型的测试用例：
  - 能测试软件所有功能的代表性测试用例
  - 专门针对可能会被修改影响的软件功能的附加测试
  - 注重于修改过的软件模块的测试

确认测试（Validation Testing）

- 确认测试标准
  - 确认测试以软件需求规约为依据，以发现软件与需求不一致的错误。主要检查软件是否实现了规约规定的全部功能要求，文档资料是否完整、正确、合理，其他的需求，如可移植性、可维护性、兼容性、错误恢复能力等是否满足
- 确认测试的结果可分为两类：
  - 满足需求规约要求的功能或性能特性，用户可以接受。
  - 发现与需求规约有偏差，此时需列出问题清单

确认测试（Validation Testing）

- 确认测试标准
  - 确认测试以软件发现软件与需求不一致的错误。主了规约规定的全部功能要求，文档、合理，其他的需求，如可移植性、错误恢复能力等是否满足
- 确认测试的结果
  - 满足需求规约要，用户可以接受。
  - 发现与需求规约问题清单



确认测试（Validation Testing）

- α测试和β测试
  - 如果软件是为一个客户开发的，那么，最后由客户进行验收测试（acceptance test），以使客户确认该软件是他 所需要的
  - 如果软件是给许多客户使用的（如市场上销售的各种软件），那么让每个客户做验收测试是不现实的。大多数软件厂商都使用一种称为α测试和β测试的过程，来发现 那些似乎只有最终用户才能发现的错误



确认测试（Validation Testing）

- α测试是由一个用户在开发者的场所进行的，软件 在开发者对用户的“指导下” 进 行测试。经α测试 后的软件称为β版软件
- β测试是由软件的最终用户在一个或多个用户场所进行的，与α测试不同，开发者通常不在测试现场，因此，β测试是软件在一个开发者不能控制的环境中 的 “活 的” 应 用 ， 用 户 记录所有在β测试中遇到 的（真正的或想象的）问题，并定期把这些问题报告给开发者，在接到β测试的问题报告后，开发者对软件进行最后的修改，然后着手准备向所有的用户发布最终的软件产品

确认测

- α测试
- β测试



系统测试（System Testing）

- 系统测试是对整个基于计算机的系统进行的一系列测试
- 系统测试的种类很多，每种测试都有不同的目的，它们从不同的角度测试计算机系统是否被正常地集成，并完成相应的功能
- 常用的系统测试包括：
  - 恢复测试（recovery testing）
  - 安全测试（security testing）
  - 压力测试（stress testing）
  - 性能测试（performance testing）

恢复测试（recovery testing）

- 恢复测试是通过各种手段，强制软件发生故障，然后来验证系统能否在指定的时间间隔内恢复正常，包括修正错误并重新启动系统
- 如果恢复是由系统自身来完成的，那么，需验证重新初始化、检查点机制、数据恢复和重启动等的正确性
- 如果恢复需要人工干预，那么要估算平均修复时间MTTR（mean time to repair）是否在用户可以接 受的范围内

安全测试（security testing）

- 安全测试用来验证集成在系统中的保护机制能否实际保护系统不受非法侵入
- 在安全测试过程中，测试者扮演一个试图攻击系统的角色，采用各种方式攻击系统。例如，截取或码译密码；借助特殊软件攻击系统；“制服”系统，使他人 无法访问；故意导致系统失效，企图在系统恢复之机 侵入系统；通过浏览非保密数据，从中找出进入系统 的钥匙等等
- 一般来说，只要有足够的时间和资源，好的完全测试一定能最终侵入系统。系统设计者的任务是把系统设计成：攻破系统所付出的代价大于攻破系统后得到信息的价值

安全测试（se

- 安全测试用来
- 在安全测试过
- 一般来说，只



机制能否实际

式图攻击系统的
截取或码译密
假”系统，使
企图在系统恢
数据，从中找出

好的完全测试
任务是把系统设
破系统后得到信

息的价值

压力测试（stress testing）

- 压力测试也称强度测试，它是在一种需要非正常数量、频率或容量的方式下执行系统，其目的是检查系统对非正常情况的承受程度。例如：
  - 当系统的中断频率是每秒1或2个时，执行每秒10个中断 的测试用例
  - 将输入数据的数量提高一个数量级来测试输入功能如何响应
  - 执行需要最大内存或其它资源的测试用例
  - 执行可能导致大量磁盘驻留数据的测试用例

压力测试（

- 压力测试也 需要非正常数量、频率 ，其目的是检查系统对 例如：
  - 当系统的中 每秒10个中断 的测试用
  - 将输入数据 输入功能如何响应
  - 执行需要最 例
  - 执行可能导 例



性能测试（performance testing）

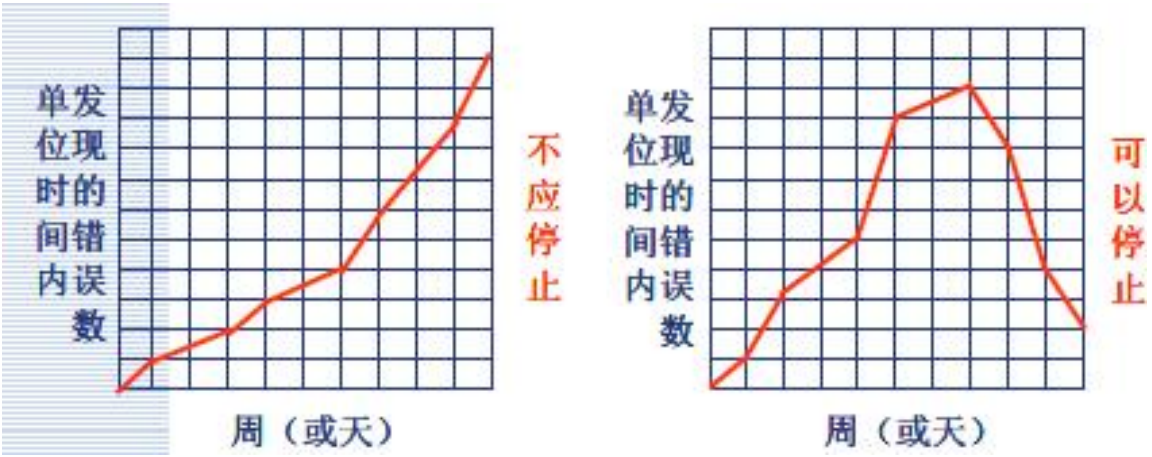
- 性能测试用来测试软件在集成的系统中的运行性能。它对实时系统和嵌入式系统尤为重要。
- 性能测试可以发生在测试过程的所有步骤中
  - 单元测试时，主要测试一个独立模块的性能，如算法的执行速度。
  - 软件集成后，进行软件整体的性能测试。
  - 计算机系统集成后，进行整个计算机系统的性能测试。
- 性能测试常常需要与压力测试结合起来进行，而且常常需要一些硬件和软件测试设备，以监测系统的运行情况

测试完成的标准

- 因为无法判定当前查出的错误是否是最后一个错误，所以决定什么时候停止程序测试就成了最困难的问题，但是测试最后一定要停止的
- 几种实用的测试完成标准：
  - Musa和Ackerman提出了一个基于统计标准的答复：
    “ 不，我们不能绝对地认定软件永远也不会再出错，但 是相对于一个理论上合理的和在试验中有效的统计模型
    来说，如果一个在按照概率的方法定义的环境中，1000个CPU小时内不出错运行的概率大于0.995的话，那么我们就有95%的信心说，我们已经进行了足够的测试”

测试完成的标准

- 使用指定的测试用例设计方法产生测试用例，运行这些测试用例均未发现错误（包括发现错误后已被纠正的情况），则测试可终止
- 观察测试阶段中单位时间内发现错误数目的曲线：

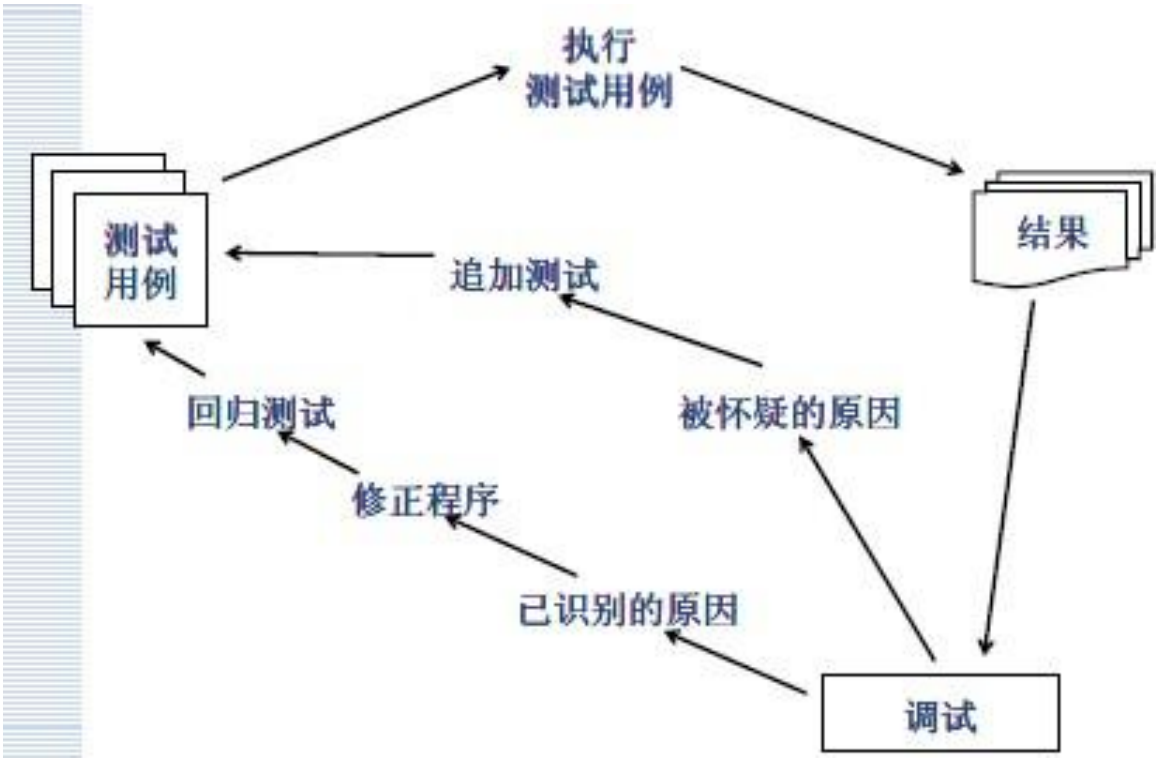


调试（Debugging）

- 测试的目的是发现错误，调试（也称排错）的目的是确定错误的原因和准确位臵，并加以纠正



调试过程

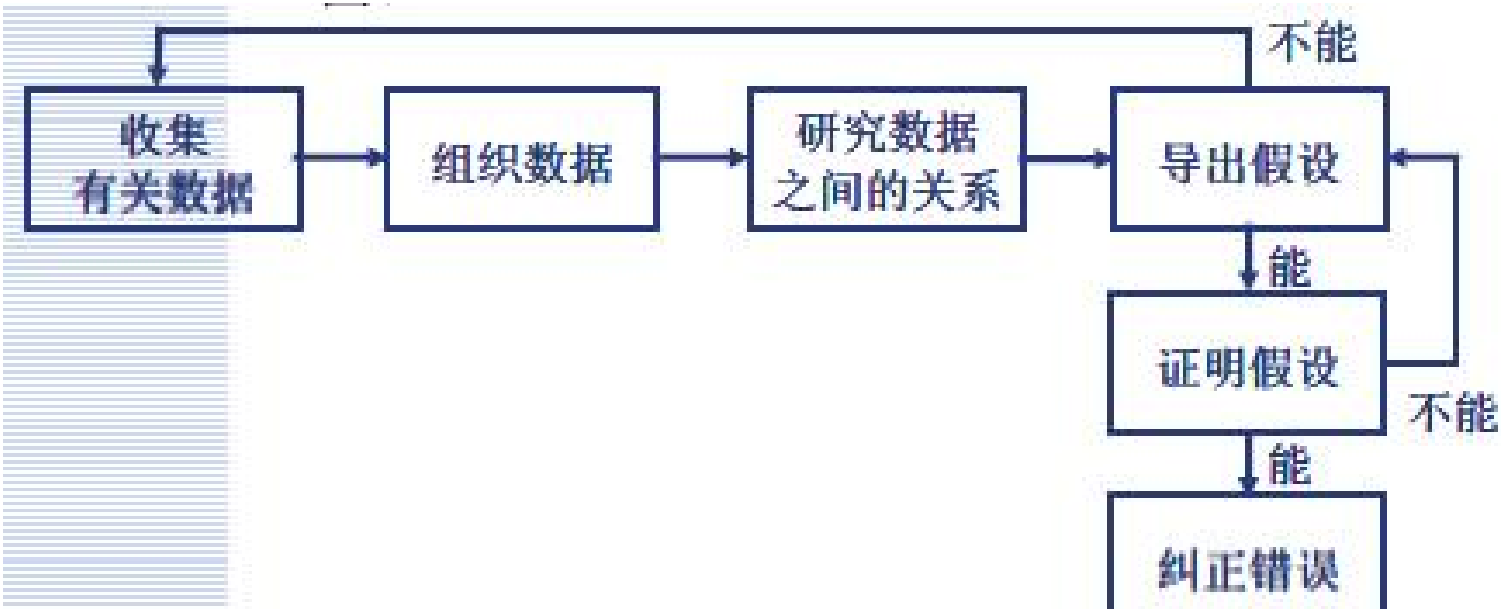


调试方法

- 蛮力法
  - 蛮力法是一种最省脑筋但又最低效的方法。它通过在程 序中设路断点，输出寄存器、存储器的内容，打印有关 变量的值等手段，获取大量现场信息，从中找出错误的 原因。
  - 这种方法效率低，输出的信息大多是无用的，通常在其 他调试方法未能找到错误原因时，才使用这种方法。
  - 可以采用二分法来逐步缩小出错的范围
- 回溯法
  - 回溯法是从错误的征兆出发，人工沿着控制流程往回跟 踪，直至发现错误的根源。这种方法适用于小型程序， 对大型程序，由于回溯的路径太多，难以彻底回溯

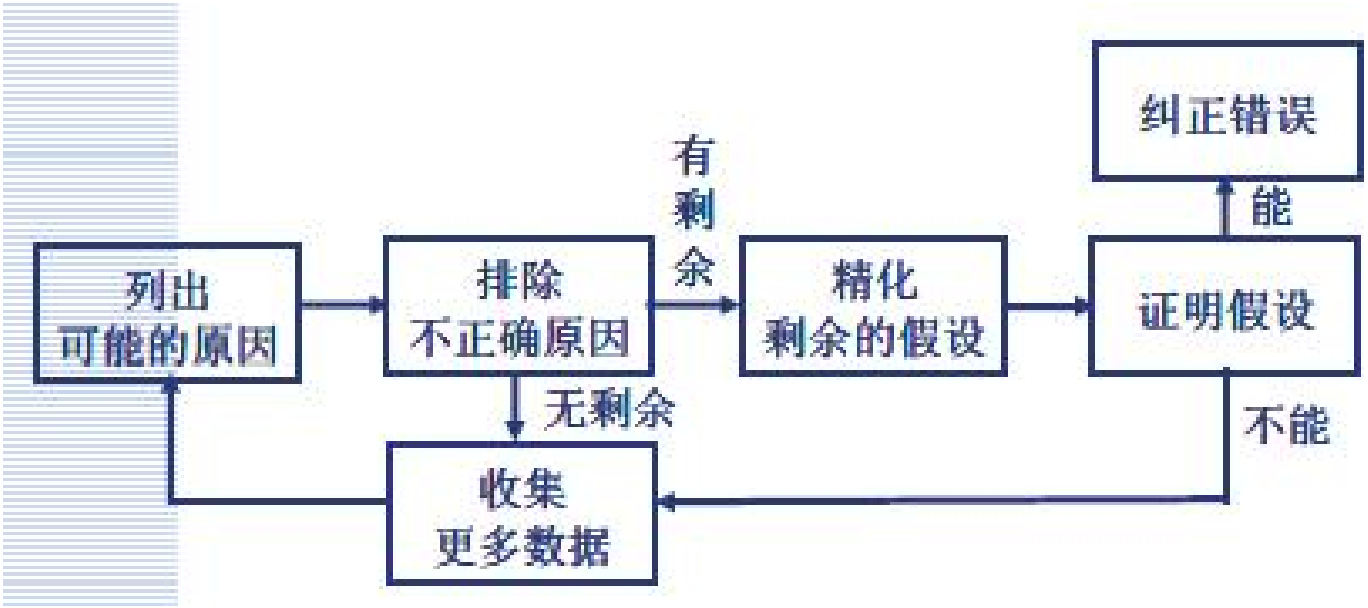
调试方法

- 原因排除法:又可分为归纳法和演绎法
  - 归纳法是一种从特殊推断一般的系统化思考方法。其基本思想是:从一些线索（错误征兆）着手，通过分析它 们之间的关系来找出错误的原因



调试方法

- 演绎法
  - 演绎法从一般原理或前提出发，假设所有可能出错的原因，排除不可能正确的假设，最后推导出结论



调试方法

- 纠正错误
  - 修改一个错误常常会引入新的错误在为纠正某个 错误而修改程序之前应该回答三个问题：
    - 在程序的其他地方是否也存在同类的错误?
    - 本次修改可能会引发什么新的错误?
    - 为了防止这个错误，我们应该做什么?

调试案例

对一个长度为100000条指令的程序进行集成测试期间记录如下面的数据:
(A)7月1日集成测试开始没有发现错误
(B)8月2日总共改正了100个错误,此时MTTF=0.4H
(C)9月1日:总共改正300个错误,此时MTTF=2H.
根据上述数据完成下列各题:
(1)估计程序中的错误总数
(2)为使MTTF达到10H必须测试和调试这个程序多长时间?

详细解答：
（1）据估算平均无故障时间MTTF的公式有：
100000/(k(Et-100))=0.4
100000/(k(Et-300))=2
得：K＝1000，Et=350 即程序中的错误总数达350。
（2）当MTTF＝10h时，有
100000/ (K(350- Ec))=10
得：Ec=340.
按七八月分测试改错进度估算，还需进行2个月的集成测试。

调试案例

对一个包含10000条机器指令的程序进行一个月集成测试后,总共改正了15个错误,此时MTTF= 10h;经过两个月测试后,总共改正了25个错误(第二个月改正了10个错误) ,MTTF= 15h。

要求:

(1)根据上述数据确定MTTF与测试时间之间的函数关系,画出MTTF与测试时间τ的关系曲线。在画这条曲线时做了什么假设?

(2)为做到MTTF= 100h,必须进行多长时间的集成测试?当集成测试结束时总共改正了多少个错误,还有多少个错误潜伏在程序中?

已知It = 10000

1.t = 24 * 30h

2.MTTF = 10000/k(ET - 15) = 10

3.t= 24 * 60h

4.MTTF = 10000/k(ET - 25) = 15

解得:

K = 100/3

ET = 45

- 已知MTTF，t的俩组数据
- 可设MTTF = at + b
- 10 = 24 * 30 a + b
- 15 = 24 * 60 a + b
- a = 1/144
- b = 5
- MTTF = 1/144 + 5

100 = t/144 + 5

t = 13680h = 19(month)

MTTF = 10000/(100/3)[45-Ec(t)] = 100

Ec(t) = 42(个)

还有54-42=3个错误存在

调试案例

如对一个长度为100000条指令的程序进行集成测试期间记录下下面的数据:
(a)7月1日:集成测试开始,没有发现错误。
(b) 8月2日:总共改正100个错误，此时MTTF=0.4h
©9月1日:总共改正300个错误,
此时,MTTF=2h根据_上列数据完成下列各题。
(1)估计程序中的错误总数。
(2)为使MTTF达到10h,必须测试和调试这个程序多长时间?
(3)画出MTTF和测试时间τ之间的函数关系曲线。

(1)
t = 24 * 31h
MTTF = 10000/K(ET - 100) = 0.4h
t = 24 * 62h
MTTF = 10000/K(ET - 300) = 2h
解得:
K = 1000
ET = 350
(2)
10 = 1/465t - 186/155
t = 5208(h)
(3)
MTTF = at + b
0.4 = (24*31)a + b
2 = (24 * 62)a + b
解得:
a = 1/465
b = 186/155
MTTF = 1/465t - 186/155

调试案例

在测试-一个长度为24000条指令的程序时,第一个月由甲,乙两名测试员各自独立测试这个程序。经一个月测试后,甲发现并改正20个错误,使MTTF达到10h。与此同时,乙发现24个错误,其中6个甲也发现了。以后由甲一个人继续测试这个程序。问:

(1)刚开始测试时程序中总共有多少个潜藏的错误?

(2)为使MTTF达到60h,必须再改正多少个错误?还需用多长测试时间?

(3)画出MTTF与集成测试时间τ之间的函数关系曲线。

t = 24 * 30

假设甲乙二人标记错误的方法一样

2a/ET = b/24

解得:

ET = 80

MTTF = 24000/K(80-20) = 10

解得:

K = 40

MTTF = 24000/(40[80-Ec(t)])

解得:

Ec(t) = 70

MTTF = 1/x = 1/M/T = T/M = 60

T/70 = 60

解得:

T = 4200(h)

4200 - 24*30 = 3480

MTTF. =at + b

60 = 4200a + b

10 = (24*30)a + b

a = 5/348

b = 360/29

MTTF = 5/387 * t + 300/29

一只骡子与软件测试员的故事……

有一个农夫决定买一匹骡子，他认为这个骡子至少得能扛动3袋大米，他才会决定买这匹骡子（这相当于用户提出的性能需求）。结果他来到农贸集市上，试了好几头骡子，都不合适，最后终于有一头骡子能够比较轻松的扛动这3袋大米，而且还潇洒的走了几步（这相当于于性能测试通过）。

农夫想看看这头骡子到底能拉多少大米，于是一袋袋的往骡子身上加，加到第7袋的时候，骡子双腿打颤，卖骡子的心疼起来，立刻制止，农夫满意的买下了这头骡子。（容量测试通过）

然后农夫高高兴兴地牵着这头骡子回家，而且给它扛了4袋大米（相当于让系统超负荷运行），因为他跑了太远才买到了这匹不可多得的骡子，他想看看它到底能有多强，所以农夫决定，让这匹骡子就扛着这四袋大米走回家试试看。（在超负荷情况下检验系统能正常运行多久，进入压力测试）

这匹骡子真的很厉害，刚开始的时候还一颠一跑的，可是家离集市有5公里，骡子越驮越费劲。快到家的时候，已经是走两步歇一步了，终于到家了。（压力测试通过）

农夫非常自豪地叫出自己的老婆，说：“老婆子，快来看看，看我买到了一头多么厉害的骡子啊！”老婆出来后，农夫把他和骡子在一路上的经历都告诉了老太婆，谁知这个老太婆却说：“你真蠢，这么大老远的路，也不让骡子驮着你，竟然和这头傻骡子一样走回来！”农夫听了，觉得非常后悔，说：“那好吧，既然在路上它没有驮我，那就让它现在补上，也算是对我的补偿。”

骡子还没有反应过来，就看那老农夫一个箭步，跳到了骡子背上（这相当于容量测试的极限点），可怜的骡子，无论如何也不会想到，这狠心的农夫竟然在它走了这么久之后，不但没有帮它卸掉身上的重担，更没有给它喝口水，竟然变本加厉的跳到了它那本已弯曲的背上。

可怜的骡子啊，就这么一命呜呼了！就看见那个骡子、农夫和4袋麦子一起轰然倒地。（相当于已经到了系统的最大拐点，造成了系统瘫痪，无法使用，容量测试结束）。

谢谢！